

---

## Bölüm-2

# SINIFLARA VE NESNELERE GİRİŞ

# OBJECTIVES



**Bu bölümde;**

- ▶ **Bir sınıf tanımlayacaksınız ve onu bir nesne oluşturmak için kullanacaksınız.**
- ▶ **Bir sınıfın davranışlarını metodlar ile uygulayacaksınız.**
- ▶ **Bir sınıfın niteliklerini örnek değişkenler (instance variables) ile uygulayacaksınız.**
- ▶ **Görevlerini gerçekleştirmelerini sağlamak için bir nesnenin yöntemlerini çağıracaksınız.**
- ▶ **Bir yöntemin yerel değişkenlerinin bir sınıfın örnek değişkenlerinden nasıl farklı olduğunu öğreneceksiniz.**
- ▶ **İlkel tiplerin ve referans tiplerinin ne olduğunu öğreneceksiniz.**
- ▶ **Bir nesnenin verilerini ilklendirmek için bir yapıcı (constructor) kullanacaksınız.**
- ▶ **Sınıfların neden gerçek dünyadaki şeyleri ve soyut varlıkları modellemenin doğal bir yolu olduğunu öğreneceksiniz.**



- ▶ Giriş
- ▶ Instance Variables: set ve get metotları
- ▶ Instance Variables: default ve explicit ilklendirme
- ▶ Account Sınıfı Örneği: Nesneleri Constructor ile ilklendirme
- ▶ Account Sınıfı için “balance” parametresi ekleme
- ▶ CASE STUDY-1: Card Shuffling
- ▶ CASE STUDY-2: Class GradeBook





- ▶ Javanın ve OOP çekirdeği sınıflardır. Javada her şey bir sınıfa ait olmalıdır.
- ▶ Sık kullandığınız bazı sınıflar;
  - ▶ `System.out` sınıfı (çıkııı verme)?
  - ▶ `Scanner` sınıfı (kullanıcı veri girişı)?
- ▶ Bu bölümde ise;

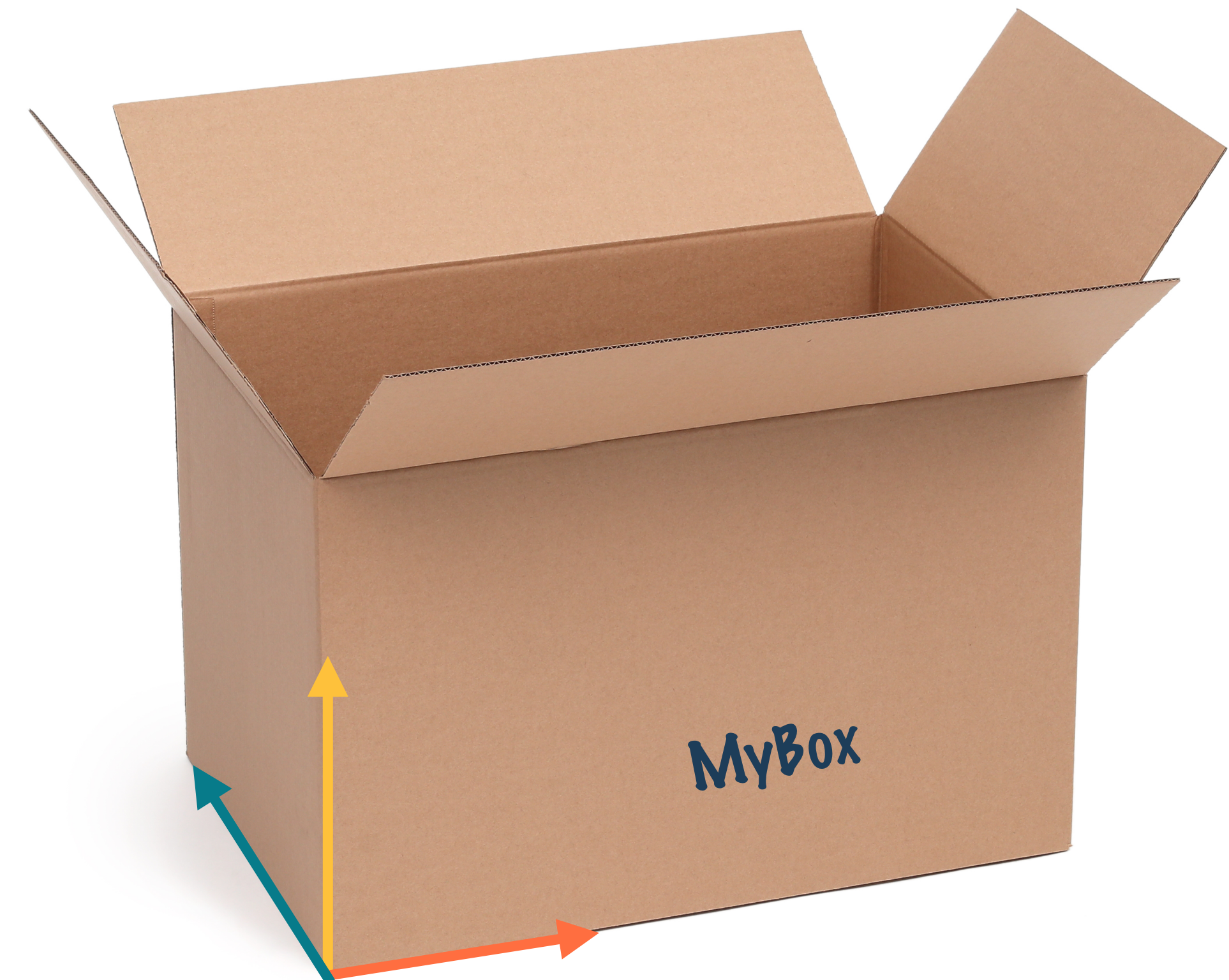
**It is time to create your own classes...**



# A SIMPLE CLASS



- ▶ Box adında bir sınıf tanımlayalım.
- ▶ 3 instance variable olsun width, height, depth.
- ▶ Bu sınıf, Box adında bir veri tipi oluşturur.
- ▶ Öncelikle taslak oluşturuldu, bu sınıfa bir varlık vermek için;
- ▶ `Box myBox = new Box();` // Böylelikle bir Box nesnesi oluştu.
- ▶ `myBox` Box sınıfının bir örneğidir.
- ▶ `myBox` kendi width, height ve depth değişkenlerine sahip.
- ▶ Bu değişkenlere erişim için: `myBox.width`



# GOOD PROGRAMMING PRACTICE

---



**Programlama yaparken isimlendirmeye dikkat!**

**Sınıf isimleri büyük harfle başlar. (PascalCase)**

**Metot ve değişken isimleri de küçük harfle başlar.  
(camelCase)**

**Sınıf içinde üst kısma instance variable yazılır.**





- ▶ Her sınıf; nesneler üretmek ve değişkenler tanımlamak için yeni bir tip olarak yaratılır.
- ▶ Sınıf nesne için bir taslaktır (template), nesne ise sınıfın bir örneğidir. (instance)
- ▶ Sınıf tanımlanırken tam olarak formu ve doğası da belirtilir.
  - ▶ İçinde bulunacak veriler, yürütülecek işlemler, ...
- ▶ Sınıfta tanımlanan değişkenlere “instance variable” denir.
- ▶ **Metodlar** sınıfın verilerinin nasıl kullanılacağına karar verir.
- ▶ Tüm metodlar ve değişkenler sınıfın birer üyesidir.
- ▶ Her sınıfta `main()` olmak zorunda değil, `main()` sadece programı başlatmak için kullanılacak.



- ▶ Bir banka uygulaması için
  - ▶ “Account” ve “AccountTest” sınıfları
- ▶ Sınıf “public class” ile tanımlanır.
- ▶ Sınıf adı, içinde bulundu dosya ile aynı olmalıdır.
- ▶ “name” (instance variable)
  - ▶ Nesne yaratıldığında değişkenler de yaratılır.
  - ▶ private olması sadece Account sınıfından erişilebileceği anlamına gelir.
  - ▶ Diğer sınıflar ise set ve get metotları ile değişkene müdahale edebilir.

```
1 // Fig. 7.1: Account.java
2 // Account class that contains a name instance variable
3 // and methods to set and get its value.
4
5 public class Account {
6     private String name; // instance variable
7
8     // method to set the name in the object
9     public void setName(String name) {
10         this.name = name; // store the name
11     }
12
13     // method to retrieve the name from the object
14     public String getName() {
15         return name; // return value of name to caller
16     }
17 }
```

**Fig. 7.1** | Account class that contains a name instance variable and methods to set and get its value.



- ▶ Bir sınıfa ait non-static metotlara "instance methods" adı verilir.
- ▶ Örneğin; setName metodu,
  - ▶ String tipinde bir parametre alır.
  - ▶ geri dönüş değeri yoktur. (void)
  - ▶ hesabın ismini tutan "name" değişkenine (instance variable) değer atar.
  - ▶ Metot içinde aynı isimle bir yerel değişken olursa (LINE 9) metot onu dikkate alır.
  - ▶ Yerel değişken, örnek değişkeni gölgeler.
  - ▶ Bu sebeple "this" anahtar kelimesi ile instance variable işaret edilir.

```
1 // Fig. 7.1: Account.java
2 // Account class that contains a name instance variable
3 // and methods to set and get its value.
4
5 public class Account {
6     private String name; // instance variable
7
8     // method to set the name in the object
9     public void setName(String name) {
10         this.name = name; // store the name
11     }
12
13     // method to retrieve the name from the object
14     public String getName() {
15         return name; // return value of name to caller
16     }
17 }
```

**Fig. 7.1** | Account class that contains a name instance variable and methods to set and get its value.



# GOOD PROGRAMMING PRACTICE

---



**Yerel değişken ismi ile örnek değişken ismi farklı olsa *this* anahtar kelimesine gerek kalmazdı. Fakat *this* kullanımı, tanımlayıcı isimleri en aza indirmek için kabul edilen bir pratiktir.**





- ▶ `getName` metodu,
- ▶ Bir `Account` nesnesinin ismini döndürür.
- ▶ parametresi yoktur.
- ▶ hesabın ismini tutan `"name"` değişkeninin (instance variable) değerini döndürür.
- ▶ Burada `"this"` anahtar kelimesini kullanmamıza gerek yok.

```
1 // Fig. 7.1: Account.java
2 // Account class that contains a name instance variable
3 // and methods to set and get its value.
4
5 public class Account {
6     private String name; // instance variable
7
8     // method to set the name in the object
9     public void setName(String name) {
10         this.name = name; // store the name
11     }
12
13     // method to retrieve the name from the object
14     public String getName() {
15         return name; // return value of name to caller
16     }
17 }
```

**Fig. 7.1** | Account class that contains a name instance variable and methods to set and get its value.





- ▶ Account sınıfını kullanabilmek için bir sürücü sınıfa ihtiyaç vardır.
- ▶ “Main()” metodu bulunan bir AccountTest sürücü sınıfı oluşturulur.
- ▶ Araba -> “sağa dön”, “daha hızlı git” gibi komutlar ile sürülür.
- ▶ AccountTest sınıfı da main() yöntemi ile Account nesnesini sürdürür. (sürücü sınıfı)
- ▶ AccountTest sınıfını ve ana yöntemini AccountTest.java dosyasında tanımlanır.
- ▶ main() yürütülmeye başladığında, AccountTest'in main() yöntemi bir Account nesnesi oluşturur ve onun getName ve setName yöntemlerini çağırır.

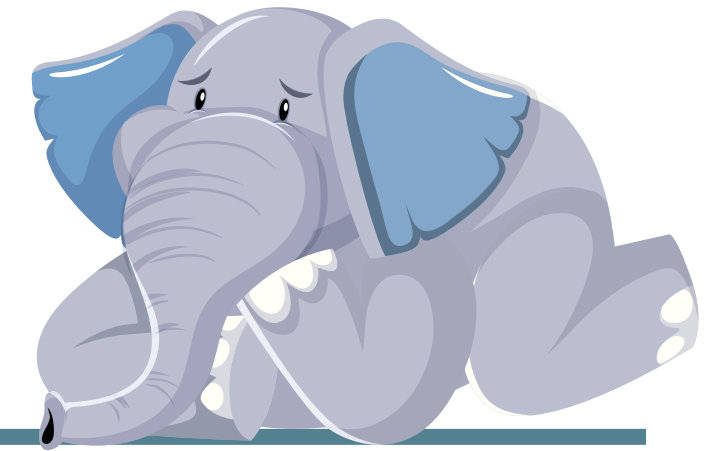


```
1 // Fig. 7.2: AccountTest.java
2 // Creating and manipulating an Account object.
3 import java.util.Scanner;
4
5 public class AccountTest {
6     public static void main(String[] args) {
7         // create a Scanner object to obtain input from the command window
8         Scanner input = new Scanner(System.in);
9
10        // create an Account object and assign it to myAccount
11        Account myAccount = new Account();
12
13        // display initial value of name (null)
14        System.out.printf("Initial name is: %s\n\n", myAccount.getName());
15    }
```

**Fig. 7.2** | Creating and manipulating an Account object. (Part 1 of 3.)

# ERROR PREVENTION TIP

---



**System.out.printf() yöntemini kullanıcıdan alınan stringler için kullanmayın. PRINTF string-format işlemi kötü niyetli kullanıcılar tarafından manipüle edilebilir.**

**NEDEN?**





```
1 // Fig. 7.2: AccountTest.java
2 // Creating and manipulating an Account object.
3 import java.util.Scanner;
4
5 public class AccountTest {
6     public static void main(String[] args) {
7         // create a Scanner object to obtain input from the command window
8         Scanner input = new Scanner(System.in);
9
10        // create an Account object
11        Account myAccount = new Account();
12
13        // display initial value of myAccount
14        System.out.printf("Initial value of myAccount is: %s\n", myAccount.getName());
15    }
```

**Fig. 7.2** | Creating and manipulating an Account object. (Part 1 of 3.)

```
16        // prompt for and read name
17        System.out.println("Please enter the name:");
18        String theName = input.nextLine(); // read a line of text
19        myAccount.setName(theName); // put theName in myAccount
20        System.out.println(); // outputs a blank line
21
22        // display the name stored in object myAccount
23        System.out.printf("Name in object myAccount is:%n%s%n",
24                           myAccount.getName());
25    }
26 }
```

**Fig. 7.2** | Creating and manipulating an Account object. (Part 2 of 3.)



# SHORT-QUESTIONS

---



- 1- Neden doğrudan “name” değişkene değer atamak yerine getName() fonksiyonunu kullandık?
- 2- getter-setter avantajı nedir?



```
1 // Fig. 7.2: AccountTest.java
2 // Creating and manipulating an Account object.
3 import java.util.Scanner;
4
5 public class AccountTest {
6     public static void main(String[] args) {
7         // create a Scanner object to obtain input from the command window
8         Scanner input = new Scanner(System.in);
9
10        // create an Account object and assign it to prompt for and read name
11        Account myAccount = new Account(); System.out.println("Please enter the name:");
12        String theName = input.nextLine(); // read a line of text
13        // display initial value of name (null) myAccount.setName(theName); // put theName in myAccount
14
15    }
```

Fig. 7.2 |

Initial name is: null

Please enter the name:

**Jane Green**

Name in object myAccount is:

Jane Green

**Fig. 7.2** | Creating and manipulating an Account object. (Part 3 of 3.)

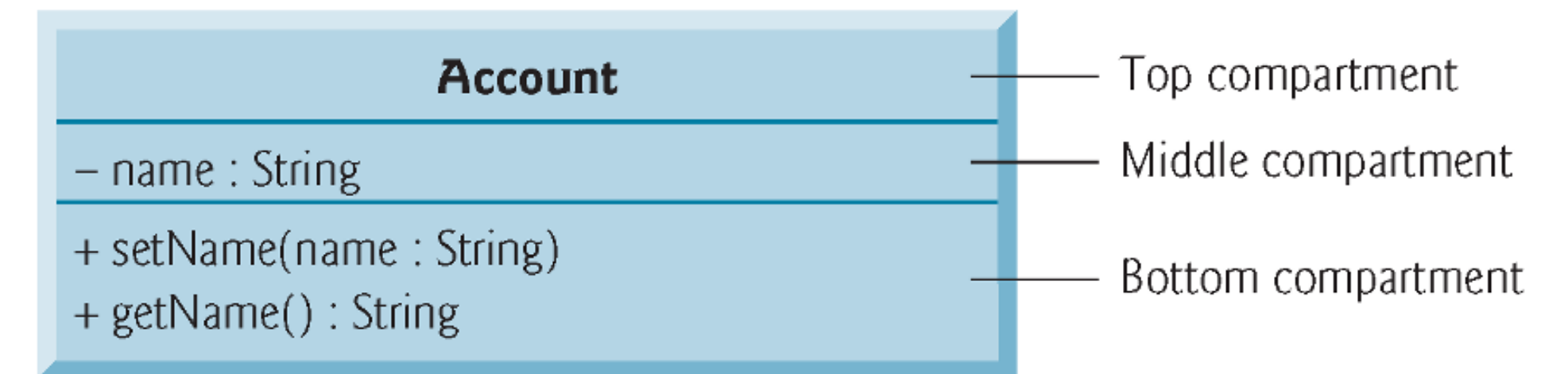


- ▶ `Account.java` ve `AccountTest.java` dosyaları aynı dizinde olmalıdır.
- ▶ Java kodlarının önce derlenmesi gerekir.
  - ▶ `javac Account.java AccountTest.java` VEYA `javac *.java`
- ▶ Sonra sürücü sınıf çalıştırılır
  - ▶ `java AccountTest.java`





- ▶ Sınıflara ait özellikler ve metotların görselleştirilmesinde UML diagramlar kullanılır.
- ▶ UML (Unified Modelling Language)
- ▶ UML diyagramları, programlamadan önce, sistemi grafiksel olarak ortaya çıkarmaya yardımcı olur.
- ▶ Üst bölme sınıf ismini içerir.
- ▶ Orta bölme, örnek değişkenleri barındırır.
- ▶ Alt bölme, constructor ve metotları içerir.
  - ▶ (-) private
  - ▶ (+) public
  - ▶ method() : Return\_Type



UML class diagram for class Account of Fig. 7.1.

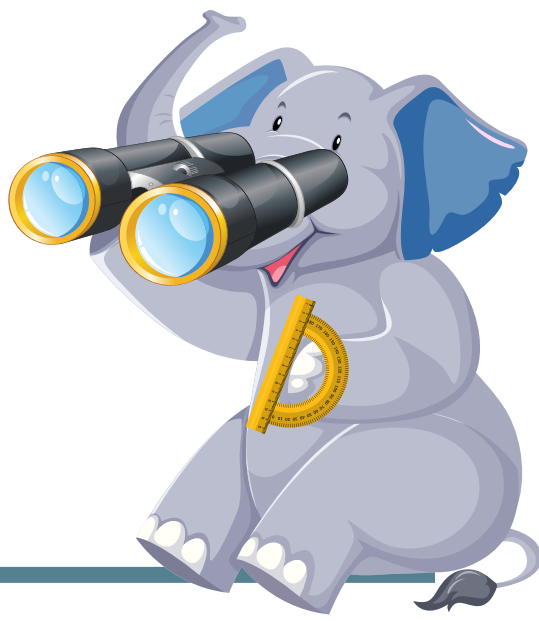




- ▶ Bir static metot, diğer static metotları ve static değişkenleri (aynı sınıftaki) doğrudan çağırabilir.
- ▶ Static metotlara nesne kullanmadan erişilebilir.
- ▶ Bir static metot, sınıfın örnek değişkenlerine ve metotlarına erişmek için nesne kullanılmalıdır.
- ▶ Java programlarında kullanılan sınıflar çoğunlukla açıkça (explicitly) import edilir.
- ▶ Aynı dizinde olan sınıflar, DEFAULT PACKAGE adı verilen aynı paket içinde derlenirler.
  - ▶ Bunlar dolaylı olarak (implicitly) import edilir. (Açıkça import gerekli değil.)
- ▶ Eğer bir sınıf açıkça import edilmeyecekse, tam nitelikli sınıf adı ile çağırılır.
- ▶ Şekil 7.2 de;
  - ▶ `Scanner input = new Scanner(System.in)` // yukarıda import edilmişse.
  - ▶ `java.util.Scanner input = new java.util.Scanner(System.in);` // import edilmemişse.

# SOFTWARE ENGINEERING OBSERVATION

---



**Bir sınıf adı her kullanıldığında tam nitelikli sınıf adı belirtilirse, Java derleyicisi ilgili Java kaynak kodu dosyasında “import” bildirimi istemez.**

**Ancak çoğu Java programcısı, daha kısa ve öz programlama için “import” bildirimini kullanmayı tercih eder.**

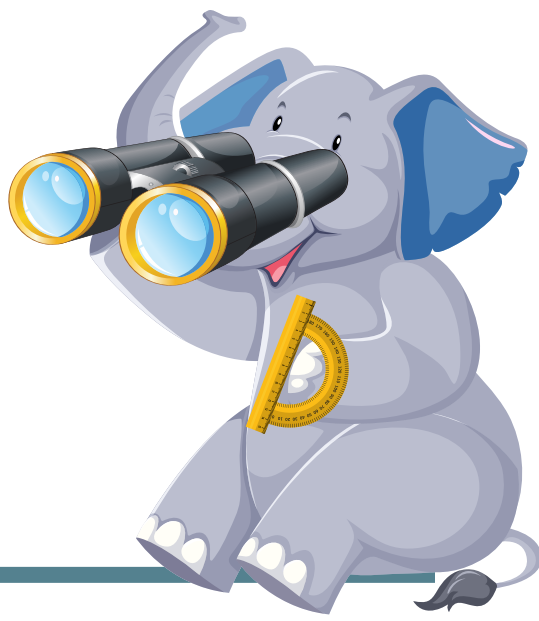


- ▶ get ve set metotları private veri üzerinde bir kontrole sahip olursunuz.
- ▶ SET metodu kullanıcının geçersiz veri girmesini engeller
  - ▶ Bir vücut sıcaklık değerinin negatif olması
  - ▶ Mart ayının günlerinin 1-31 değerleri dışında olması
  - ▶ Bir ürüne şirketin katalog listesinde olmayan bir ürün kodu verilmesi
- ▶ GET metodu ise veriyi farklı şekilde sunma imkanı verir.
  - ▶ Örneğin, "not" 1-100 arasında bir değer alırken, "getNot()" ile harfli değerler (AA, BB, ...) geri dönebilir.
- ▶ Bu yöntemler sayesinde daha hatalar azalır, sağlamlık ve güvenlik artar.
- ▶ Örnek değişkenlerin "private" erişim denetleyici (access modifier) ile bildirilmesine BİLGİ GİZLEME denir.
  - ▶ Account sınıfındaki "name" değişkeni kapsüllenmiştir. (ENCAPSULATED)



# SOFTWARE ENGINEERING OBSERVATION

---



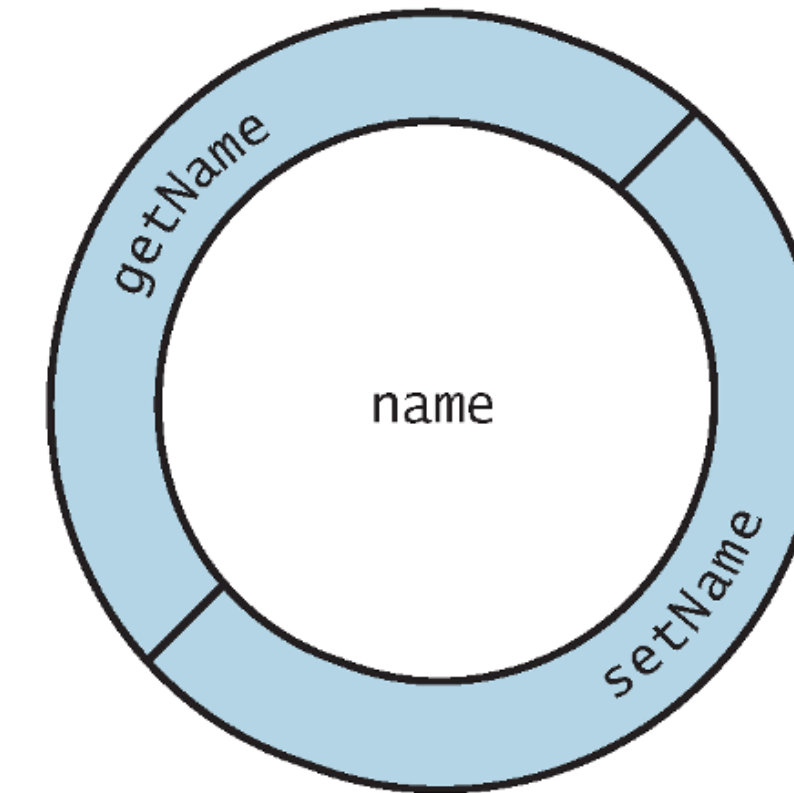
**Her örnek değişken ve yöntem bildirimi bir erişim denetleyici ile başlar.**

**Genel olarak, örnek değişkenler “private” ve yöntemler “public” olarak bildirilmelidir.**





- ▶ “name” private bir örnek değişken olduğu için, doğrudan erişim sağlanamaz.
- ▶ Değişkenin nasıl kapsüllendiği görselde verilmiştir.
- ▶ Java da primitive tipler için default ilk değerler;
  - ▶ byte, char, short, long, int, float, double, => 0
  - ▶ boolean => false
  - ▶ string => null
- ▶ Sınıf tanımlamasında ayrıca ilk değer atanabilir.
  - ▶ `private String name = “Deniz”`
- ▶ ilk değer atama için **CONSTRUCTOR** kullanılabilir.



**Fig. 7.4** | Conceptual view of an Account object with its encapsulated private instance variable name and protective layer of public methods.



- ▶ Account sınıf için bir constructor (yapıcı) eklendi.
- ▶ Constructor ismi sınıf ismi ile aynı olmalı.
- ▶ Constructor geri dönüş değeri olmaz.
- ▶ Constructor eklenmezse java derleyicisi default bir constructor (parametresiz) sağlar. Örnek değişkenlere ait default değerler set edilir.
- ▶ Constructor eklenmişse, derleyici default constructor yaratmaz.

```
1 // Fig. 7.5: Account.java
2 // Account class with a constructor that initializes the name.
3
4 public class Account {
5     private String name; // instance variable
6
7     // constructor initializes name with parameter name
8     public Account(String name) { // constructor name is class name
9         this.name = name;
10    }
11
12    // method to set the name
13    public void setName(String name) {
14        this.name = name;
15    }
16
17    // method to retrieve the name
18    public String getName() {
19        return name;
20    }
21 }
```

**Fig. 7.5** | Account class with a constructor that initializes the name.



- ▶ Account sınıf için bir constructor (yapıcı) eklendi.
- ▶ Constructor ismi sınıf ismi ile aynı olmalı.
- ▶ Constructor geri dönüş değeri olmaz.
- ▶ Constructor eklenmezse java derleyicisi default bir constructor (parametresiz) sağlar. Örnek değişkenlere ait default değerler set edilir.
- ▶ Constructor eklenmişse, derleyici default constructor yaratmaz.

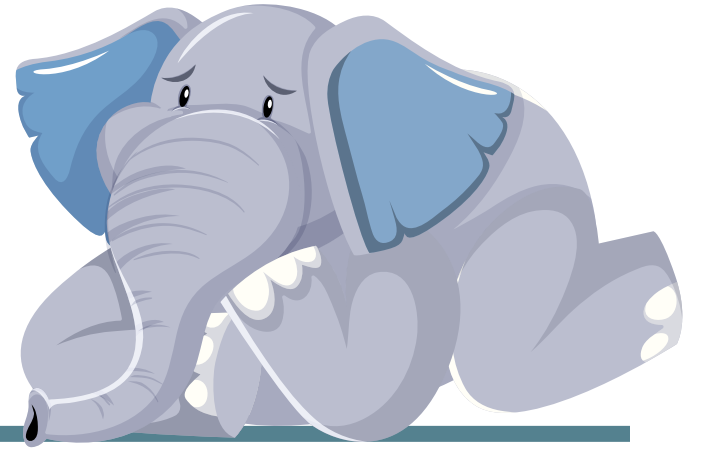
```
1 // Fig. 7.5: Account.java
2 // Account class with a constructor that initializes the name.
3
4 public class Account {
5     private String name; // instance variable
6
7     // constructor initializes name with parameter name
8     public Account(String name) { // constructor name is class name
9         this.name = name;
10    }
11
12    // method to set the name
13    public void setName(String name) {
14        this.name = name;
15    }
16
17    // method to retrieve the name
18    public String getName() {
19        return name;
20    }
21 }
```

**Fig. 7.5** | Account class with a constructor that initializes the name.



# ERROR PREVENTION TIP

---



**Her ne kadar mümkün olsa da  
Constructor içinde metod çağırmayınız.**





- ▶ AccountTest sınıfında iki Account Nesnesi oluşturuldu.
- ▶ Değişkenlere getter method ile ulaşıldı.

```

1 // Fig. 7.6: AccountTest.java
2 // Using the Account constructor to initialize the name instance
3 // variable at the time each Account object is created.
4
5 public class AccountTest {
6     public static void main(String[] args) {
7         // create two Account objects
8         Account account1 = new Account("Jane Green");
9         Account account2 = new Account("John Blue");
10
11         // display initial value of name for each Account
12         System.out.printf("account1 name is: %s\n", account1.getName());
13         System.out.printf("account2 name is: %s\n", account2.getName());
14     }
15 }

```

```

account1 name is: Jane Green
account2 name is: John Blue

```

**Fig. 7.6** | Using the Account constructor to initialize the name instance variable at the time each Account object is created.

```

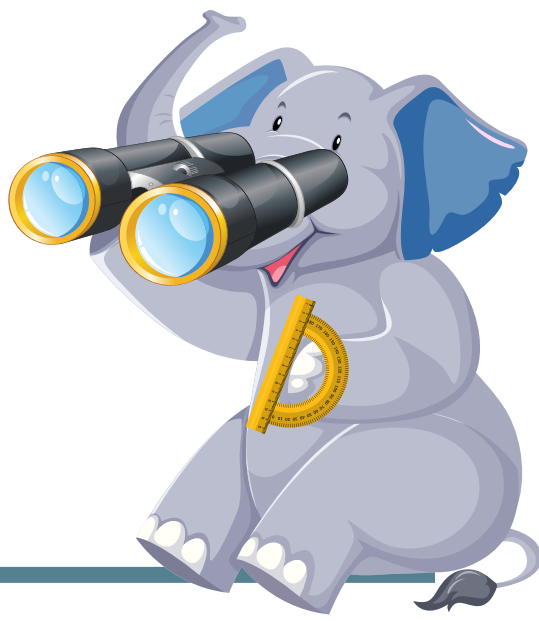
1 // Fig. 7.5: Account.java
2 // Account class with a constructor that initializes the name.
3
4 public class Account {
5     String name; // instance variable
6
7     // constructor initializes name with parameter name
8     Account(String name) { // constructor name is class name
9         name = name;
10    }
11
12    // to set the name
13    void setName(String name) {
14        name = name;
15    }
16
17    // to retrieve the name
18    String getName() {
19        return name;
20    }
21 }

```

Account class with a constructor that initializes the name.

# SOFTWARE ENGINEERING OBSERVATION

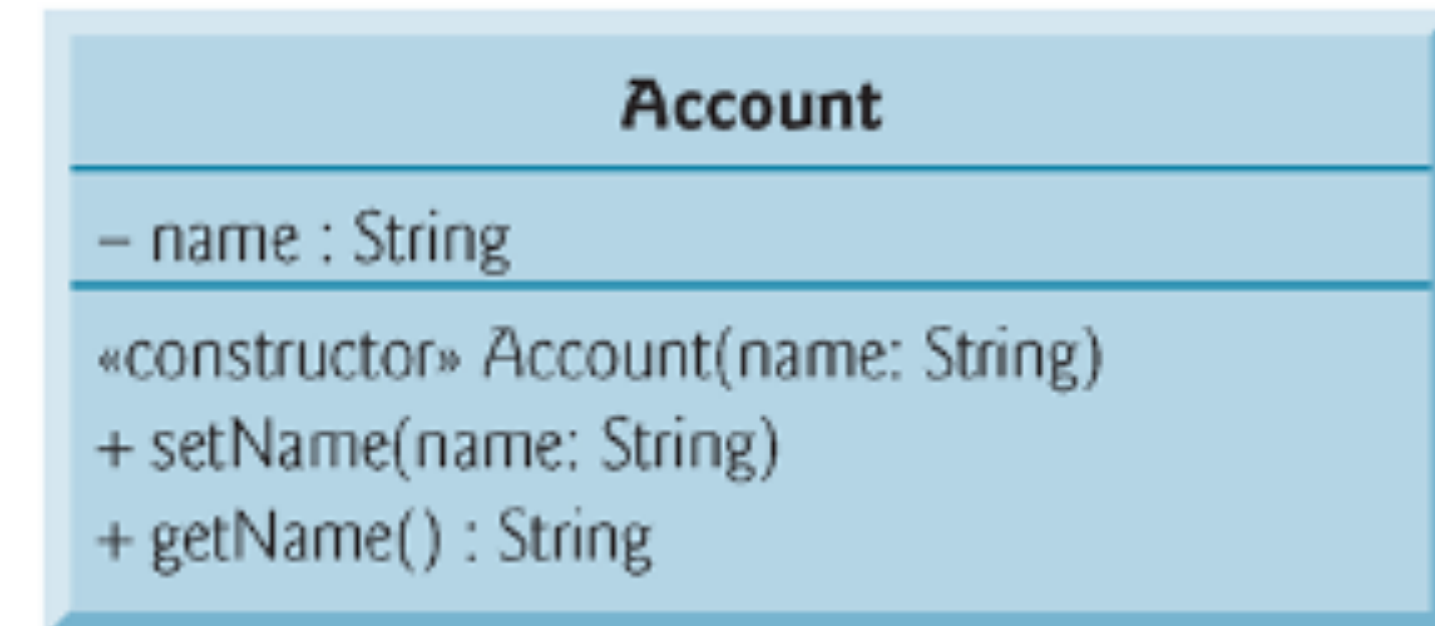
---



**Sınıfa ait örnek değişkenleri default olarak setlemek mümkün olsa da, özel constructor; örnek değişkenlere yeni bir nesne ile anlamlı değerler atamayı garantiler.**



- ▶ Diagramın 3. Bölmesine constructor eklenir.
- ▶ << constructor >> şeklinde yazılır.
- ▶ Metot ile benzer şekilde kullanılır.



UML class diagram for Account class of Fig. 7.5.





- ▶ Banka hesabının isminin yanına yeni değişken eklenir. Double "balance".
- ▶ Constructor için de yeni bir parametre eklendi.
- ▶ Doğrudan değer atamak yerine constructor kullanmak geçerli değerlerin atanmasını sağlamaktadır.

```
1 // Fig. 7.8: Account.java
2 // Account class with a double instance variable balance and a constructor
3 // and deposit method that perform validation.
4
5 public class Account {
6     private String name; // instance variable
7     private double balance; // instance variable
8
9     // Account constructor that receives two parameters
10    public Account(String name, double balance) {
11        this.name = name; // assign name to instance variable name
12
13        // validate that the balance is greater than 0.0; if it's not,
14        // instance variable balance keeps its default initial value of 0.0
15        if (balance > 0.0) { // if the balance is valid
16            this.balance = balance; // assign it to instance variable balance
17        }
18    }
19 }
```

**Fig. 7.8** | Account class with a double instance variable balance and a constructor and deposit method that perform validation. (Part 1 of 3.)





- ▶ Banka hesabının isminin yanına yeni değişken eklenir. Double "balance".
- ▶ Constructor için de yeni bir parametre eklendi.
- ▶ Doğrudan değer atamak yerine constructor kullanmak geçerli değerlerin atanmasını sağlamaktadır.
- ▶ Hesaba para yatırıldığında balance değeri güncellenir. (Deposit() )
- ▶ **getBalance()** ile değer geri döner.

```
1 // Fig. 7.8: Account.java
2 // Account class with a double instance variable balance and a constructor
3 // and deposit method that perform validation.
4
5 public class Account {
6     private String name; // instance variable
```

```
20 // method that deposits (adds) only a valid amount to the balance
21 public void deposit(double depositAmount) {
22     if (depositAmount > 0.0) { // if the depositAmount is valid
23         balance += depositAmount; // add it to the balance
24     }
25 }
```

```
26
27 // method returns the account balance
28 public double getBalance() {
29     return balance;
30 }
31
```

**Fig. 7.8** | Account class with a double instance variable balance and a constructor and deposit method that perform validation. (Part 2 of 3.)



- ▶ AccountTest sınıfında constructor ile iki nesne oluşturulur.
- ▶ Name ve balance değişkenleri setlenir.

```
1 // Fig. 7.9: AccountTest.java
2 // Inputting and outputting floating-point numbers with Account objects.
3 import java.util.Scanner;
4
5 public class AccountTest {
6     public static void main(String[] args) {
7         Account account1 = new Account("Jane Green", 50.00);
8         Account account2 = new Account("John Blue", -7.53);
9
10        // display initial balance of each object
11        System.out.printf("%s balance: $%.2f\n",
12            account1.getName(), account1.getBalance());
13        System.out.printf("%s balance: $%.2f\n\n",
14            account2.getName(), account2.getBalance());
15    }
```

**Fig. 7.9** | Inputting and outputting floating-point numbers with Account objects. (Part 1 of 4.)



- ▶ AccountTest sınıfında constructor ile iki nesne oluşturulur.
- ▶ Name ve balance değişkenleri setlenir.
- ▶ Hesaba para yatırma işlemi yapılır.
- ▶ Ekran double değer basma;
  - ▶ .2f -> virgülden sonra 2 basamak.
  - ▶ .f -> virgülden sonra 1 basamak

```
1 // Fig. 7.9: AccountTest.java
2 // Inputting and outputting floating-point numbers with Account objects.
3 import java.util.Scanner;
4
```

```
31 System.out.print("Enter deposit amount for account2: "); // prompt
32 depositAmount = input.nextDouble(); // obtain user input
33 System.out.printf("%nadding %.2f to account2 balance%n\n",
34     depositAmount);
35 account2.deposit(depositAmount); // add to account2 balance
36
37 // display balances
38 System.out.printf("%s balance: $.2f%n",
39     account1.getName(), account1.getBalance());
40 System.out.printf("%s balance: $.2f%n\n",
41     account2.getName(), account2.getBalance());
42 }
43 }
```

Part 1 of 4.)

**Fig. 7.9** | Inputting and outputting floating-point numbers with Account objects. (Part 3 of 4.)





### ► Çıktılar;

- OBJ-1: name and balance valid.
- OBJ-2: name OK, balance invalid. (balance=?)
- Hesaplara para eklendi, yeni değerler OK.

```
Jane Green balance: $50.00
John Blue balance: $0.00

Enter deposit amount for account1: 25.53

adding 25.53 to account1 balance

Jane Green balance: $75.53
John Blue balance: $0.00

Enter deposit amount for account2: 123.45

adding 123.45 to account2 balance

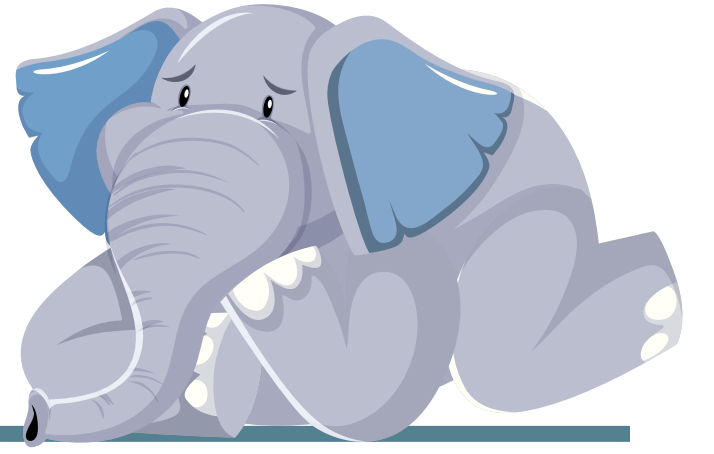
Jane Green balance: $75.53
John Blue balance: $123.45
```

**Fig. 7.9** | Inputting and outputting floating-point numbers with Account objects. (Part 4 of 4.)



# ERROR PREVENTION TIP

---

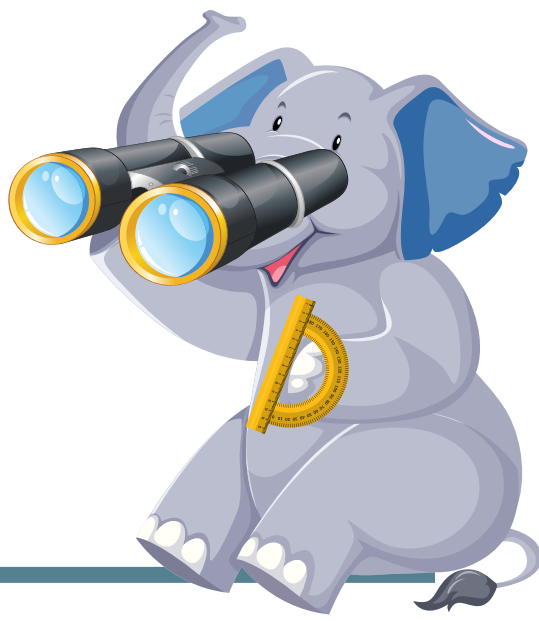


**İlk değeri atanmamış bir değişkeni kullandığınıza derleme hatası alırsınız. Böylece tehlikeli mantık hatalarında korunursunuz.**

**Çalışma zamanında bu hatayı almak yerine, derlemede almak her zaman daha iyidir.**

# SOFTWARE ENGINEERING OBSERVATION

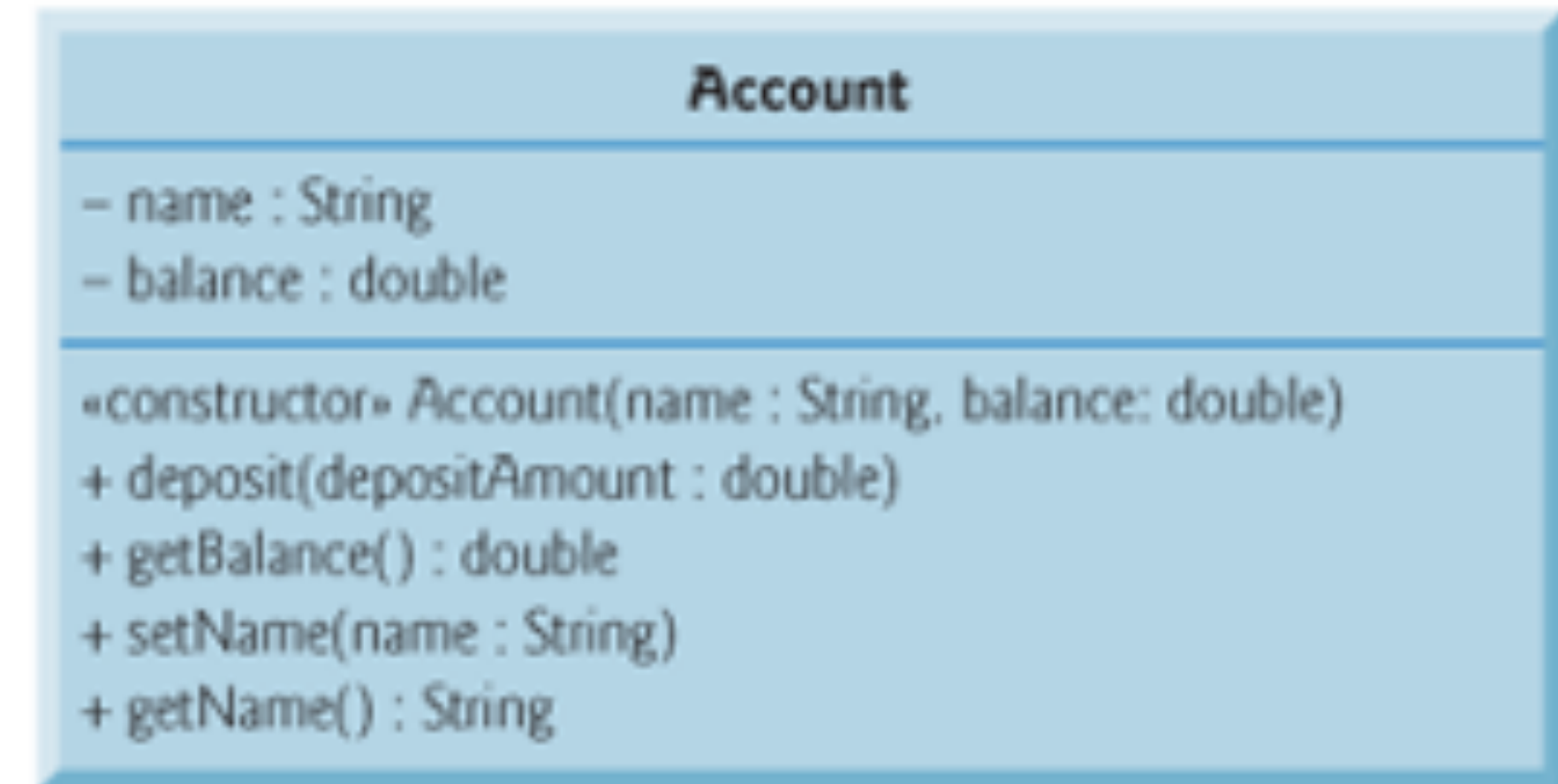
---



**Tekrar edilen kodları, kodun bir kopyasını içeren bir metot çağrısı ile değiştirmek programın büyüklüğünü azaltır ve sürdürülebilirliği geliştirir.**



- ▶ Diagramın 2. Bölmesine
  - ▶ Yeni örnek değişken eklenir (PRIVATE)
- ▶ Diagramın 2. Bölmesine
  - ▶ Yeni constructor eklenir.
  - ▶ Yeni metotlar eklenir.



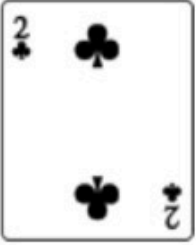
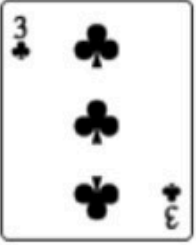
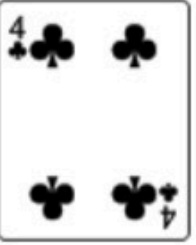
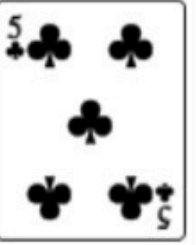
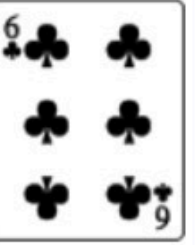
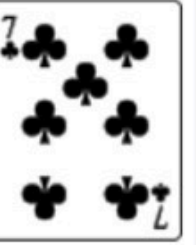
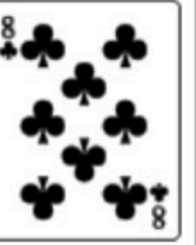
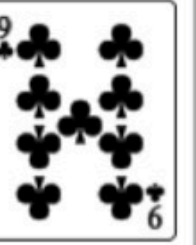
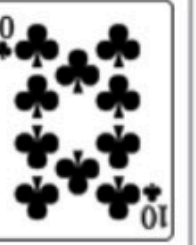
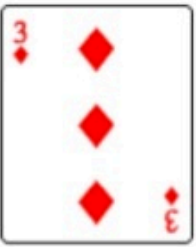
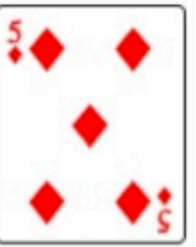
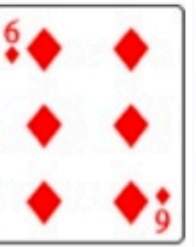
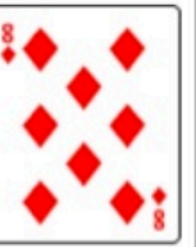
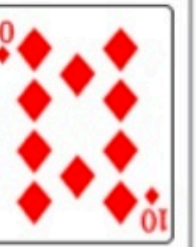
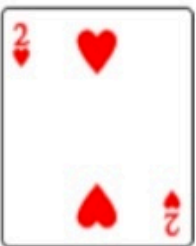
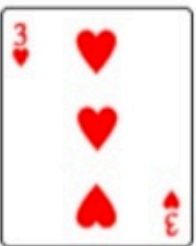
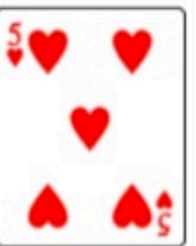
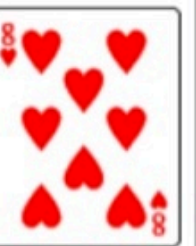
| UML class diagram for Account class of Fig. 7.8.





- İlk olarak tüm kartları içerebilen bir Card sınıfı üretilecek.
- 52 lik deste oluşturulacak.
- Test sınıfı ile card karıştırma işlemi gösterilecek.

Example set of 52 playing cards; 13 of each suit clubs, diamonds, hearts, and spades

|          | Ace                                                                                 | 2                                                                                    | 3                                                                                     | 4                                                                                     | 5                                                                                     | 6                                                                                     | 7                                                                                     | 8                                                                                     | 9                                                                                     | 10                                                                                    | Jack                                                                                  | Queen                                                                                 | King                                                                                  |
|----------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Clubs    |   |   |   |   |   |   |   |   |   |   |   |   |   |
| Diamonds |  |  |  |  |  |  |  |  |  |  |  |  |  |
| Hearts   |  |  |  |  |  |  |  |  |  |  |  |  |  |
| Spades   |  |  |  |  |  |  |  |  |  |  |  |  |  |





- ▶ Card sınıfı “face” ve “suit” olarak iki String örnek değişkene sahiptir.
- ▶ Face: Ace, 2-10, jack, queen, king.
- ▶ (Yüz: As, 2-10, vale, kız, papaz)
- ▶ Suit: Hearts, Spades, Diamonds, Clubs.
- ▶ (Takım: Kupa, Maça, Karo, Sinek)
- ▶ toString() ile iki değer birleştirilip geriye döner.

```
1 // Fig. 7.11: Card.java
2 // Card class represents a playing card.
3
4 public class Card {
5     private final String face; // face of card ("Ace", "Deuce", ...)
6     private final String suit; // suit of card ("Hearts", "Diamonds", ...)
7
8     // two-argument constructor initializes card's face and suit
9     public Card(String cardFace, String cardSuit) {
10         this.face = cardFace; // initialize face of card
11         this.suit = cardSuit; // initialize suit of card
12     }
13
14     // return String representation of Card
15     public String toString() {
16         return face + " of " + suit;
17     }
18 }
```

**Fig. 7.11** | Card class represents a playing card.



- ▶ DeckOfCard sınıfı;
- ▶ randomNumbers (STATIC)
- ▶ NUMBER\_OF\_CARDS (STATIC)
- ▶ deck
- ▶ currentCard

```
1 // Fig. 7.12: DeckOfCards.java
2 // DeckOfCards class represents a deck of playing cards.
3 import java.security.SecureRandom;
4
5 public class DeckOfCards {
6     // random number generator
7     private static final SecureRandom randomNumbers = new SecureRandom();
8     private static final int NUMBER_OF_CARDS = 52; // constant # of Cards
9
10    private Card[] deck = new Card[NUMBER_OF_CARDS]; // Card references
11    private int currentCard = 0; // index of next Card to be dealt (0-51)
12
```

**Fig. 7.12** | DeckOfCards class represents a deck of playing cards. (Part 1 of 4.)



► DeckOfCard sınıfı;

► randomNumbers (STATIC)

► NUMBER\_OF\_CARDS (STATIC)

► deck

► currentCard

► deckofCards()

```
1 // Fig. 7.12: DeckOfCards.java
2 // DeckOfCards class represents a deck of playing cards.
3 import java.util.*;
4 import java.security.*;
```

```
13 // constructor fills deck of Cards
14 public DeckOfCards() {
15     String[] faces = {"Ace", "Deuce", "Three", "Four", "Five", "Six",
16                     "Seven", "Eight", "Nine", "Ten", "Jack", "Queen", "King"};
17     String[] suits = {"Hearts", "Diamonds", "Clubs", "Spades"};
18
19     // populate deck with Card objects
20     for (int count = 0; count < deck.length; count++) {
21         deck[count] =
22             new Card(faces[count % 13], suits[count / 13]);
23     }
24 }
25
```

new SecureRandom();  
constant # of Cards

/ Card references  
to be dealt (0-51)

1 of 4.)

**Fig. 7.12** | DeckOfCards class represents a deck of playing cards. (Part 2 of 4.)





## DeckOfCard sınıfı;

randomNumbers (STATIC)

NUMBER\_OF\_CARDS (STATIC)

deck

currentCard

deckofCards()

shuffle()

```
13 // co
14 publi
15 St
16
17 St
18
19 //
20 fo
21
22
23 }
24 }
25
```

```
26 // shuffle deck of Cards with one-pass algorithm
27 public void shuffle() {
28     // next call to method dealCard should start at deck[0] again
29     currentCard = 0;
30
31     // for each Card, pick another random Card (0-51) and swap them
32     for (int first = 0; first < deck.length; first++) {
33         // select a random number between 0 and 51
34         int second = randomNumbers.nextInt(NUMBER_OF_CARDS);
35
36         // swap current Card with randomly selected Card
37         Card temp = deck[first];
38         deck[first] = deck[second];
39         deck[second] = temp;
40     }
41 }
42
```

Fig. 7.12 | DeckOfCards class represents a deck of playing cards. (Part 3 of 4.)

Fig. 7.12 | DeckOfCards class represents a deck of playing cards. (Part 1 of 4.)

Fig. 7.12 | DeckOfCards class represents a deck of playing cards. (Part 2 of 4.)



# GOOD PROGRAMMING PRACTICE

---



## SWAP işlemi:

| Deck-1   | Deck-2   | Temp |
|----------|----------|------|
| Sinek-As | Kupa-kız | -    |
| Kupa-kız | Kupa-kız |      |

| Deck-1   | Deck-2   | Temp     |
|----------|----------|----------|
| Sinek-As | Kupa-kız | -        |
| Sinek-As | Kupa-kız | Sinek-As |
| Kupa-kız | Kupa-kız | Sinek-As |
| Kupa-kız | Sinek-As | Sinek-As |

# SHORT-QUESTIONS

---



ÖRNEK KOD

“Fisher-Yates shuffling algorithm”



## DeckOfCard sınıfı:

randomNumbers (STATIC)

NUMBER\_OF\_CARDS (STATIC)

deck

currentCard

deckofCards()

shuffle()

dealCard()

```
26 // shuffle deck of Cards with one-pass algorithm
27 public void shuffle() {
28     // next call to method dealCard should start at deck[0] again
```

```
43 // deal one Card
44 public Card dealCard() {
45     // determine whether Cards remain to be dealt
46     if (currentCard < deck.length) {
47         return deck[currentCard++]; // return current Card in array
48     }
49     else {
50         return null; // return null to indicate that all Cards were dealt
51     }
52 }
53 }
```

**Fig. 7.12** | DeckOfCards class represents a deck of playing cards. (Part 4 of 4.)

**Fig. 7.12** | DeckOfCards class represents a deck of playing cards. (Part 1 of 4.)

**Fig. 7.12** | DeckOfCards class represents a deck of playing cards. (Part 2 of 4.)





- ▶ DeckOfCardsTest sınıfında demo yapılır.
- ▶ Önce bir nesne üretilir.
- ▶ Sonra kartlar karıştırılır.
- ▶ Sonra 4'er li olarak ekrana basılır.

|                   |                   |                   |                  |
|-------------------|-------------------|-------------------|------------------|
| Six of Spades     | Eight of Spades   | Six of Clubs      | Nine of Hearts   |
| Queen of Hearts   | Seven of Clubs    | Nine of Spades    | King of Hearts   |
| Three of Diamonds | Deuce of Clubs    | Ace of Hearts     | Ten of Spades    |
| Four of Spades    | Ace of Clubs      | Seven of Diamonds | Four of Hearts   |
| Three of Clubs    | Deuce of Hearts   | Five of Spades    | Jack of Diamonds |
| King of Clubs     | Ten of Hearts     | Three of Hearts   | Six of Diamonds  |
| Queen of Clubs    | Eight of Diamonds | Deuce of Diamonds | Ten of Diamonds  |
| Three of Spades   | King of Diamonds  | Nine of Clubs     | Six of Hearts    |
| Ace of Spades     | Four of Diamonds  | Seven of Hearts   | Eight of Clubs   |
| Deuce of Spades   | Eight of Hearts   | Five of Hearts    | Queen of Spades  |
| Jack of Hearts    | Seven of Spades   | Four of Clubs     | Nine of Diamonds |
| Ace of Diamonds   | Queen of Diamonds | Five of Clubs     | King of Spades   |
| Five of Diamonds  | Ten of Clubs      | Jack of Spades    | Jack of Clubs    |

Fig. 7.13 | Card shuffling and dealing. (Part 2 of 2.)

```
1 // Fig. 7.13: DeckOfCardsTest.java
2 // Card shuffling and dealing.
3
4 public class DeckOfCardsTest {
5     // execute application
6     public static void main(String[] args) {
7         DeckOfCards myDeckOfCards = new DeckOfCards();
8         myDeckOfCards.shuffle(); // place Cards in random order
9
10        // print all 52 Cards in the order in which they are dealt
11        for (int i = 1; i <= 52; i++) {
12            // deal and display a Card
13            System.out.printf("%-19s", myDeckOfCards.dealCard());
14
15            if (i % 4 == 0) { // output a newline after every fourth card
16                System.out.println();
17            }
18        }
19    }
20 }
```

Fig. 7.13 | Card shuffling and dealing. (Part 1 of 2.)





- ▶ Sınıftaki öğrencilerin notları hakkında bilgi veren bir uygulama.
- ▶ Notları ekranda göster;
  - ▶ Notlar
  - ▶ Sınıf ortalaması
  - ▶ En düşük not
  - ▶ En yüksek not
  - ▶ Not dağılım şeması



- ▶ Sınıftaki öğrencilerin notları hakkında bilgi veren bir uygulama.
- ▶ String courseName
- ▶ int [] grades
- ▶ Constructor Gradebook(String courseName, int [] grades)
- ▶ Setter and getter
- ▶ processGrades() //tüm bilgileri getirecek.
- ▶ outputGrades() //notları getirir.
- ▶ getAverage(), getMinimum(), getMaximum()
- ▶ outputBarChart() //grafik oluşturma



- ▶ Kendi sınıflarımızı yaratmayı öğrendik.
- ▶ Bu sınıflara ait nesneler yaratmayı öğrendik.
- ▶ Bu nesneleri yönetmek için gerekli metotların kullanımını öğrendik.
- ▶ Yerel değişkenler ile örnek değişkenler arasındaki farkı öğrendik.
- ▶ Sadece örnek değişkenlere otomatik olarak değer atanacağını öğrendik.
- ▶ Constructor kullanımını ve bunların ilk değer atamadaki önemini öğrendik.
- ▶ Kabaca bir UML sınıf diagramını üretmeyi öğrendik.
- ▶ 3 farklı konuda uygulamalar gerçekleştirdik.